05 — Team Lead, GitHub, Integration and Deployment Guide

Audience: Member 1 (Team Lead) — this is your personal implementation handbook, in the same spirit as Documents 2, 3 and 4 are for Members 2, 3 and 4. Everything here must match Document 1 (`docs/01-MASTER-PROJECT-DEVELOPMENT-GUIDE.md`) exactly — if anything here ever seems to disagree with Document 1, Document 1 wins and this file is wrong and must be fixed.

Important note up front: this guide assumes you are *also* learning Git, GitHub and deployment for the first time — being "the lead" here means owning the repo, the integration points, and the final release, not being an expert the other three are waiting on. Every command below is explained, not just listed. If a term is unfamiliar the first time it appears, there's a short explanation right there — you shouldn't need to leave this document to look something up.

---

1. Role recap

As Member 1, you own: GitHub repository administration (creating it, inviting Members 2/3/4, setting permissions and branch protection); the `docs/` folder and `README.md`; the three integration-critical backend services — `server/src/services/priceComparison.js`, `server/src/services/priceMonitor.js`, and `server/src/services/alertEvaluator.js` (coordinate with Member 3 before changing `Alert` model fields, since they own `server/src/models/Alert.js`); reviewing every pull request that touches shared contracts (this document set, the API routes' shapes, the `Offer`/`Alert` models), auth-sensitive code, monitoring orchestration, or production configuration; daily coordination of blockers and task completion across the team; and deployment plus the final demo, including making sure live data is never presented as more verified than it actually is. Per Document 1 Section 14, every other PR needs at least one review from a teammate who isn't the author — yours specifically get reviewed where they touch shared contracts, same as anyone else's.

---

2. Day 1: Git and GitHub setup on Windows

2.1 What Git is and what GitHub is (these are two different things)

Git is a program that runs on your own computer. It keeps a complete history of every change ever made to a folder of files (a "repository," or "repo") — every commit is a saved snapshot you can always go back to. Git works entirely offline; you could use it with no internet connection at all, just tracking changes on your own laptop.

GitHub is a website (github.com) that hosts a copy of a Git repository online, and adds collaboration features Git itself doesn't have: a place everyone's local copy can push to and pull from, pull requests (a structured way to propose and review a change before it's merged), issue tracking, and permissions (who

can push, who can approve a merge). You and your three teammates will each have your own local copy of the repo (via Git) that all stays in sync through one shared copy on GitHub.

So: Git = version control on your machine. GitHub = the shared, hosted home for everyone's copies, plus the PR/review workflow.

2.2 Installing Git for Windows

Open PowerShell and check whether Git is already installed:

```
git --version
```

If that prints something like `git version 2.44.0.windows.1`, skip to Section 2.3. If PowerShell says `git is not recognized as an internal or external command`, install it one of these two ways:

Option A — winget (recommended, built into Windows 10/11):

```
winget install --id Git.Git -e --source winget
```

Close and reopen PowerShell after installation so it picks up the updated `PATH`, then run `git --version` again to confirm.

Option B — official installer: download from git-scm.com/download/win and run it. During setup, the default options are fine for this project — in particular, keep "Git from the command line and also from 3rd-party software" selected (not the "Git Bash only" option), and keep the default branch name option as `main` if asked (this project uses `main`, not `master`).

2.3 Configuring your Git identity

Every commit you make is stamped with a name and email — this is how Git (and GitHub) attribute each change to a person, both in `git log` and in the PR/commits view on GitHub. Set this once, globally, so every repo on your machine uses it:

```
git config --global user.name "Nagavamshi Yamjala"
  git config --global user.email "vamshisrinivas20@gmail.com"
```

Use the same email you'll sign up to GitHub with — GitHub links commits to your GitHub profile by matching this email address, which is how your commits show your avatar and username on github.com instead of looking anonymous.

Confirm it took effect:

```
git config --global user.name
  git config --global user.email
```

Ask each of your three teammates to run the same two `git config --global` commands (with their own name/email) on their own machines on Day 1 — it's a two-line step but easy to forget, and a commit made with no identity set is awkward to fix after the fact.

2.4 Creating a GitHub account

If you don't already have one, go to github.com/join and sign up — a free account is all this project needs (everything in this guide uses GitHub's free tier). Use a real email you check regularly, since GitHub sends PR notifications and collaborator invites there. Make sure each of Members 2, 3 and 4 also has a GitHub account and tells you their GitHub **username** (not just their name) before Day 1's repo setup — you'll need the exact username to invite them (Section 4).

---

3. Creating the repository

3.1 Create the repo on GitHub

1. Sign in at github.com, click the + icon top-right → **New repository**.
2. **Repository name:** `PriceRadar` (exact casing doesn't functionally matter to Git, but keep it consistent with Document 1 Section 4's structure, which uses `PriceRadar/` as the root folder name).
3. **Description (optional):** "Real-time product price comparison and price-drop alert system — 7-day team training project."
4. **Visibility — Public or Private:** choose **Private**. Reasoning: commit history permanently includes each team member's name and (often) email address, and this is a training-program project, not a public portfolio piece the team has agreed to showcase — defaulting to private avoids exposing that without an explicit decision to do so. This is **reversible later** — a private repo can be switched to public at any time from Settings → General → Danger Zone, once/if the team or training program decides they want public visibility (e.g. to link it from a resume). If your training program specifically wants the repo public for visibility or grading purposes, that's a reasonable override — just make it a deliberate choice, not the default.
5. **Initialize this repository with:** check **Add a README file**, and for **.gitignore**, choose the **Node** template from the dropdown (GitHub's built-in Node `.gitignore` already covers `node_modules/`, `.env`, build output, and common log/cache files — you'll review and extend it in Section 3.3). Leave "Choose a license" as **None** for this internal training project.
6. Click **Create repository**.

3.2 Clone it to your Windows machine

Pick (and remember) a consistent local folder for your projects — per your own setup this is `D:\Projects`. Open PowerShell:

```
cd D:\Projects\
  git clone https://github.com/<your-github-username>/PriceRadar.git
  cd PriceRadar
```

`git clone` downloads the full repository (all its history, not just the current files) into a new `PriceRadar` folder. `cd PriceRadar` moves your PowerShell session into it — run every other Git command in this guide from inside this folder unless told otherwise.
The first time you push, Git may open a browser window asking you to sign in to GitHub and authorize the Git Credential Manager (installed automatically with Git for Windows) — sign in once and it's remembered for future pushes from this machine.

3.3 Review and extend `.gitignore`

Open `.gitignore` (created by the Node template) in VS Code:

```
code .
```

Confirm it already includes `node_modules/` and `.env`. If `.env` isn't explicitly listed (some older template versions only list `.env.local` or similar), add these lines at the end to be certain — this directly backs ACC-09 from Document 1 Section 17 (no `.env` file is ever committed):

```
# Environment files — never commit real secrets
  .env
  .env.local
  server/.env
  client/.env
```

3.4 Replace the README stub

Replace the auto-generated `README.md` with a short project stub for Day 1 (you'll flesh this out further by Day 6, per Section 14 below):

```
# PriceRadar

  Real-time product price comparison and intelligent price-drop alert system.
  Built in 7 days by a 4-person team as a training-program project.

  ## Status
  Day 1 — scaffolding in progress.

  ## Documentation
  Full specifications live in [`docs/`](./docs):
  - [01 — Master Project Development Guide](./docs/01-MASTER-PROJECT-DEVELOPMENT-GUIDE.
  md)
  - [02 — Frontend and API Integration Guide](./docs/02-FRONTEND-AND-API-INTEGRATION-GU
  IDE.md)
  - [03 — Backend, Database and Authentication Guide](./docs/03-BACKEND-DATABASE-AND-AU
  THENTICATION-GUIDE.md)
  - [04 — Retailer Data, Scheduler and Notifications Guide](./docs/04-RETAILER-DATA-SCH
  EDULER-AND-NOTIFICATIONS-GUIDE.md)
  - [05 — Team Lead, GitHub, Integration and Deployment Guide](./docs/05-TEAM-LEAD-GITH
  UB-INTEGRATION-AND-DEPLOYMENT-GUIDE.md)

  ## Team
  | Member | Role |
  |---|---|
  | Member 1 | Team Lead — GitHub, integration, price comparison/monitoring/alert engin
  e, deployment |
  | Member 2 | Frontend (React + Vite) |
  | Member 3 | Backend, database, authentication |
  | Member 4 | Retailer data, scheduler, notifications |

  ## Deployed URLs
  _To be filled in on Day 6 (Section 14 of Document 5)._
```

3.5 Create the folder skeleton from Document 1 Section 4

Still inside `D:\Projects\PriceRadar`, create the full structure. Git does not track empty folders, so each folder below gets a placeholder file (`.gitkeep`) wherever it would otherwise be empty — these can be deleted later once a real file lands in that folder.

```
# client/
  New-Item -ItemType Directory -Force -Path client/public
  New-Item -ItemType Directory -Force -Path client/src/components
  New-Item -ItemType Directory -Force -Path client/src/pages
  New-Item -ItemType Directory -Force -Path client/src/services
  New-Item -ItemType Directory -Force -Path client/src/context
  New-Item -ItemType Directory -Force -Path client/src/styles

  # server/
  New-Item -ItemType Directory -Force -Path server/src/config
  New-Item -ItemType Directory -Force -Path server/src/models
  New-Item -ItemType Directory -Force -Path server/src/routes
  New-Item -ItemType Directory -Force -Path server/src/controllers
  New-Item -ItemType Directory -Force -Path server/src/middleware
  New-Item -ItemType Directory -Force -Path server/src/services/notifications
  New-Item -ItemType Directory -Force -Path server/src/providers
  New-Item -ItemType Directory -Force -Path server/src/jobs
  New-Item -ItemType Directory -Force -Path server/src/utils
  New-Item -ItemType Directory -Force -Path server/tests

  # .github/workflows/ (for the scheduler and CI workflows, Sections 12 and 12.1)
  New-Item -ItemType Directory -Force -Path .github/workflows

  # Placeholder files so Git tracks these folders before real code exists
  $placeholders = @(
    "client/public/.gitkeep", "client/src/components/.gitkeep", "client/src/pages/.gitk
  eep",
    "client/src/services/.gitkeep", "client/src/context/.gitkeep", "client/src/styles/.
  gitkeep",
    "server/src/config/.gitkeep", "server/src/models/.gitkeep", "server/src/routes/.git
  keep",
    "server/src/controllers/.gitkeep", "server/src/middleware/.gitkeep",
    "server/src/services/notifications/.gitkeep", "server/src/providers/.gitkeep",
    "server/src/jobs/.gitkeep", "server/src/utils/.gitkeep", "server/tests/.gitkeep"
  )
  foreach ($p in $placeholders) { New-Item -ItemType File -Force -Path $p | Out-Null }
```

`docs/` already exists (that's where this file and Documents 1–4 live) — if it doesn't yet in your clone, create it and add Documents 1–4 before continuing.

3.6 First commit and push to `main`

```
git status
```

Read the output — it should list the new `README.md` change, `.gitignore`, and every new folder/file you just created, all as untracked or modified. This is the habit you'll use before every commit for the rest of the project: **look at `git status` before you stage anything**, so you always know exactly what you're about to commit.

```
git add README.md .gitignore client server docs .github
  git status
  git commit -m "feat: scaffold project folder structure and README"
  git push -u origin main
```

`git add` stages the specific paths listed (not everything in the folder — see Section 6.4 for why this matters). `git commit` saves a snapshot of the staged changes with a message. `git push -u origin main` uploads that commit to GitHub's copy of `main`, and `-u` ("set upstream") remembers the link between your local `main` and GitHub's `main` so future pushes from this branch can just be `git push`. Refresh the repo page on github.com — you should see the full folder structure and the updated README.

---

4. Inviting team members and permissions

4.1 Inviting Members 2, 3 and 4

Since the repo is private, only people you explicitly invite can see or clone it.

1. On the repo page, go to **Settings** → **Collaborators** (left sidebar, under "Access").
2. Click **Add people**.
3. Enter each teammate's exact GitHub **username** or the email tied to their GitHub account, one at a time, and send the invite.
4. Each teammate receives an email/notification and must **accept** it (check their GitHub notifications bell, or the emailed link) before they can clone or push.

If the repo were public instead, anyone could read/clone it without an invite, but **writing** (pushing branches, opening PRs) still requires being added as a collaborator either way — visibility and write access are separate settings.

4.2 What role to give them

When adding each collaborator, GitHub asks for a role. Choose **Write** for Members 2, 3 and 4 — not **Admin**.

- **Write** lets them clone, push branches, open pull requests, and push commits to branches they create. This is everything they need to do their jobs.
- **Admin** would additionally let them change repository settings, delete the repo, manage other people's access, and change branch protection rules. There's no reason for anyone but you (the repo creator/lead) to hold that — Document 1 Section 14 makes you the owner of "GitHub repo, branches, PR process," and keeping Admin to yourself is what makes that ownership actually mean something rather than being nominal.

4.3 Branch protection on `main`

This prevents anyone (including you) from accidentally pushing straight to `main` and skipping review.

1. Go to **Settings** → **Branches** (left sidebar).
2. Under "Branch protection rules," click **Add branch protection rule** (or **Add rule**, depending on GitHub's current UI).
3. **Branch name pattern:** `main`.
4. Check **Require a pull request before merging**.
5. Under that, check **Require approvals**, and set the required number of approvals to **1**.
6. Leave the other options (required status checks, required conversation resolution, etc.) unchecked for now — they're reasonable additions later (e.g. once the CI workflow from Section 12.1 exists, you could require it to pass) but aren't necessary to start the project safely.
7. Click **Create** (or **Save changes**).

If this option isn't available on your account tier: branch protection rules on private repositories have historically been limited to paid plans on some GitHub pricing tiers; GitHub has also been known to extend this to free private repos at various times, so check what your account actually offers when you get here. If the "Add branch protection rule" option is greyed out, missing, or your plan doesn't support it on a private repo, the documented fallback is: **the team agrees by convention that nobody ever pushes directly to `main` — every change goes through a feature branch and a pull request, enforced by team discipline rather than by GitHub.** State this plainly in a team message (and optionally pin it in a GitHub Discussion or an issue) rather than silently skipping it; this is the same practical safety the automated rule would give you, just enforced by agreement instead of software.

---

5. Branch strategy (final, authoritative for the whole team)

This is the one authoritative branch list for the whole project — Documents 2, 3 and 4 all reference this section instead of each defining their own. Use **exactly** these names; don't invent variants.

Branch	Used by	Purpose
`main`	everyone (merge target only)	Always stable, always deployable. Nobody commits to it directly — only merges land here via reviewed PRs.
`feature/project-scaffold`	Members 1, 2, 3 (Day 1)	Initial client/server scaffolding, base config, Day-1 setup work that doesn't fit a more specific branch yet.
`feature/product-search`	Member 2 (with Member 3's API)	Search bar, search-results page, product cards, comparison/detail page (frontend).
`feature/user-authentication`	Members 2 and 3	Registration/login pages and `AuthContext` (Member 2); registration/login routes, password hashing, JWT issuing, auth middleware (Member 3).
`feature/dashboard-watchlist`	Members 2 and 3	Dashboard page and Watchlist page/components (Member 2); watchlist CRUD routes and model (Member 3).
`feature/alerts`	Members 2 and 3	Create/edit alert form and alert-management page (Member 2); alert CRUD routes and model (Member 3, coordinating with Member 1 on the fields `alertEvaluator.js` depends on — Section 9 below).
`feature/notification-settings`	Members 2, 3 and 4	Notification preferences page and notification-history page (Member 2); preference/history routes (Member 3); the Telegram link-code backend piece (Member 4).
`feature/retailer-provider`	Member 4 (reviewed by Member 1)	Provider interface, `fixture.js`, `flipkart.js`, the provider dispatcher.
`feature/price-monitor`	Member 1 (reviewed by Member 4)	`priceComparison.js`, `priceMonitor.js`, `alertEvaluator.js`, the `/api/monitor/*` routes.
`feature/email-notifications`	Member 4	Brevo integration (`services/notifications/email.js`).
`feature/telegram-notifications`	Member 4	Telegram Bot API integration (`services/notifications/telegram.js`).
`feature/deployment`	Member 1	`.github/workflows/`, Render configuration files, deployment-related README/docs updates.

`main` stays stable and integration-only — nobody, including you as lead, writes code directly on `main`. Every change, no matter how small, happens on one of the `feature/*` branches above and reaches `main` only through a reviewed, merged pull request. This is what branch protection (Section 4.3) enforces technically, and what the whole team agrees to by convention even where it can't be enforced technically.

---

6. The complete feature-branch workflow (canonical copy)

This is the one full walkthrough of the day-to-day Git workflow. Documents 2, 3 and 4 point back here instead of repeating it — if a teammate asks "how do I commit/push/open a PR," point them at this section.

6.1 Clone (once, per machine)

Already covered in Section 3.2 for your own machine. Each teammate runs the same `git clone https://github.com/<your-username>/PriceRadar.git` once they've accepted their collaborator invite (Section 4.1).

6.2 Start from an up-to-date `main`

Before starting *any* new feature, always update your local `main` first:

```
git switch main
  git pull --ff-only origin main
```

`git switch main` moves you to the `main` branch locally. `git pull --ff-only origin main` downloads any new commits from GitHub's `main` and fast-forwards your local `main` to match — `--ff-only` makes Git refuse (rather than silently create a merge commit) if your local `main` has diverged, which shouldn't happen if nobody commits directly to `main`, but it's a safe habit regardless.

6.3 Create and switch to a feature branch

```
git switch -c feature/product-search
```

`-c` creates the branch and switches to it in one step, branching off whatever commit you were on (your freshly-updated `main`). Use the exact branch name from Section 5's table.
Check which branch you're currently on at any time with:

```
git branch
```

The branch with a `*` next to it is your current one.

6.4 Do the work, then inspect before staging

After implementing and manually testing your change:

```
git status
```

This lists every file Git has noticed changed, been created, or been deleted since your last commit, split into "staged" and "not staged" sections. Read it every time — it's how you catch an accidental file (a stray config change, a debug file) before it gets committed.

```
git diff
```

This shows the actual line-by-line changes for everything *not yet staged*. Skim it before staging — it's the fastest way to catch a leftover `console.log`, a hardcoded test value, or (critically) a secret that accidentally got typed into a tracked file instead of `.env`.

Now stage **specific paths**, not everything:

```
git add server/src/services/priceComparison.js server/src/services/priceMonitor.js
```

Why not `git add -A` or `git add .` as a default habit: both of those stage *every* changed/new file in the repo, including anything you were experimenting with, a half-finished unrelated change, a stray file VS Code or a tool created, or — worst case — a `.env` file that somehow isn't correctly ignored. Naming exact paths means you only ever commit what you actually mean to, every time. (There's nothing wrong with `git add -A` used deliberately by someone who already reviewed `git status` and genuinely wants everything in it — but as the default reflex for a 4-person shared repo, naming paths is the safer habit, and it's the one this project uses consistently.)

Confirm exactly what's staged before committing:

```
git diff --cached
```

This is the same idea as `git diff`, but for what's already staged — your last chance to review precisely what this commit will contain.

6.5 Commit with a conventional commit message

```
git commit -m "feat: implement price comparison and lowest-offer selection"
```

Use this prefix convention consistently across the whole team:

Prefix	For
`feat:`	A new feature or capability
`fix:`	A bug fix
`test:`	Adding or updating tests, no behavior change
`docs:`	Documentation-only changes

Real examples from this project:

```
feat: implement price comparison and lowest-offer selection
  feat: add target-price and fixed-drop alert evaluation
  fix: prevent duplicate notifications on repeated monitor runs
  test: cover alert re-arming after price rises then drops again
  docs: update API contract for /api/monitor/status response shape
```

6.6 Push the branch

```
git push -u origin feature/product-search
```

The first push of a new branch needs `-u` (set-upstream) so Git remembers which remote branch this local branch tracks; after that, plain `git push` from this branch is enough.

6.7 Open a pull request on GitHub

Go to the repo on github.com — GitHub usually shows a yellow banner ("Compare & pull request") for a branch you just pushed; click it, or go to **Pull requests** → **New pull request** and pick your branch manually.

In the PR description, include:

- **What changed**, in one or two sentences.
- **Which task ID it corresponds to**, from the relevant role document — e.g. `Implements BE-02 (watchlist CRUD routes)` or `Implements FE-04 (search results page)` or `Implements DATA-03 (fixture provider)`, so anyone reading the PR can find the full task spec in Document 2/3/4 without guessing.
- **How you tested it** (a Postman request, a manual click-through, a `npm test` run).
- Anything the reviewer should pay special attention to (e.g. "this touches the `Alert` model — flagging for Member 1/3 coordination per Document 1 Section 14").

Request a review from the appropriate reviewer per Document 1 Section 14's ownership matrix — you (Member 1) for anything touching shared contracts, auth, monitoring, or production config; otherwise any teammate who isn't the author.

6.8 Reviewing and approving a PR

As the reviewer, open the **Files changed** tab on the PR and go through the checklist in Section 11 below. Leave inline comments on specific lines where something needs to change (GitHub lets you click the `+` next to any line in the diff). When you're satisfied, use **Review changes** → **Approve** (or **Request changes** if something must be fixed first). Per Document 1 Section 14's rule, this review step is not optional for PRs touching shared contracts, auth, monitoring, or production config — and every other PR still needs at least one approval from someone other than its author.

6.9 Merging

Once a PR has the required approval(s) and (once Section 12.1's CI exists) passing checks, merge it. GitHub offers a few merge strategies — this project uses **Squash and merge** consistently. Reasoning, explained simply: squashing combines every commit on the feature branch into one clean commit on `main`, with one message (GitHub lets you edit it — keep the conventional-commit prefix). This keeps `main`'s history readable as "one line per shipped feature" instead of a long trail of "wip," "fix typo," "actually fix it" commits from the feature branch — which matters a lot for 4 beginners trying to later understand what changed and when. Use the **Squash and merge** button (not plain "Merge" or "Rebase and merge") on every PR.

After merging, delete the feature branch from GitHub (there's a button right there after merge) — it's already fully captured in `main`'s history via the squash commit, so keeping it around just adds clutter. Your local copy of that branch can be deleted too, once you've pulled the merge (Section 6.10):

```
git branch -d feature/product-search
```

6.10 Verifying the merged code

After any merge to `main` — yours or a teammate's — update your local `main` and actually run the app before building anything else on top of it:

```
git switch main
  git pull --ff-only origin main
```

Then re-run whatever locally verifies the merged feature (`npm run dev` for both `client` and `server`, the relevant manual test, or `npm test` in `server/`). This is what Section 18 below calls out as a common failure mode if skipped: building your next feature on top of a `main` that was merged but never actually re-tested.

6.11 Updating a feature branch with changes from `main`

If `main` has moved on while you're still working on a feature branch (a teammate merged something in the meantime), bring those changes into your branch before you finish, so your PR doesn't conflict later:

```
git switch feature/your-branch-name
  git pull origin main
```

This project uses `git pull origin main` (while sitting on your feature branch) as the one consistent approach for this, rather than teaching a separate `git fetch` + `git merge main` two-step — `git pull origin main` does the fetch-and-merge in one command, which is simpler for a beginner workflow, and it has the exact same effect. This can produce a **merge conflict** if the same lines were changed on both `main` and your branch — that's normal and expected sometimes, not a sign something went wrong; resolve it with the VS Code walkthrough in Section 7 below, then continue your work.

7. Resolving merge conflicts in VS Code

A conflict happens when Git can't automatically combine two sets of changes to the same lines of a file — most often during `git pull origin main` (Section 6.11) or while merging a PR that's gone stale. Git pauses and asks you to resolve it by hand.

7.1 What a conflict looks like

Git inserts **conflict markers** directly into the affected file:

```
<<<<<<< HEAD
  const JWT_EXPIRES_IN = "7d";
  =======
  const JWT_EXPIRES_IN = "24h";
  >>>>>>> main
```

- Everything between `<<<<<<< HEAD` and `=======` is **your** current branch's version.
- Everything between `=======` and `>>>>>>> main` is the **incoming** version (from `main`, in this example).
- You need to end up with exactly one correct version and no markers left in the file.

7.2 Resolving it in VS Code

1. Open the conflicted file in VS Code (`code .` from the repo root if it's not already open). VS Code detects the conflict markers automatically and shows an inline UI right above the conflict block with clickable options: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**, **Compare Changes**.
2. Click **Accept Current Change** to keep your branch's version, **Accept Incoming Change** to take `main`'s version, or manually edit the text yourself if the correct result is a combination of both (common when both sides changed *different* things near the same lines) — after manually editing, delete the `<<<<<<<`, `=======`, and `>>>>>>>` marker lines yourself if any remain.
3. Repeat for every conflicted block VS Code highlights — the Source Control panel (left sidebar, the icon that looks like a branching line) lists every file still showing conflicts, so you can see when you've cleared them all.
4. Once a file has no conflict markers left and reads correctly, stage it:

```
git add <path/to/resolved-file>
```

5. Once every conflicted file is staged, complete the merge with a commit:

```
git commit
```

Git will have already pre-filled a merge commit message (something like `Merge branch 'main' into feature/your-branch-name`) — that default message is fine to keep as-is; just save and close the editor that opens (in VS Code, `Ctrl+Enter` or the checkmark in the Source Control panel; in a plain terminal editor, save and exit).

6. Verify the file actually works (re-run the app, re-run the relevant test) before pushing — a conflict resolved incorrectly (e.g. accidentally discarding a real change) won't show up as a Git error, only as broken behavior.

```
git push
```

---

8. Safe recovery procedures

Mistakes happen — the point of this section is to fix them without losing anyone's work. Read each situation below and use the matching command; don't reach for the destructive commands in Section 8.7 as a first resort.

8.1 Discarding an uncommitted change (after reviewing it first)

If you've edited a file and decide you don't want the change at all:

```
git diff <path/to/file>
```

Look at exactly what you're about to throw away — this step matters, because the next command is **not** reversible.

```
git restore <path/to/file>
```

This resets the file back to its last-committed state. (You may also see `git checkout -- <path/to/file>` in older tutorials — it does the same thing; `git restore` is the newer, clearer-intent command and is what this project teaches consistently, so use `git restore`.)

8.2 Unstaging a file

If you ran `git add` on something you didn't mean to stage yet:

```
git restore --staged <path/to/file>
```

This moves the file back to "changed but not staged" — it does **not** discard your edits, it only undoes the `git add`.

8.3 Recovering a committed-but-not-yet-pushed change

If you committed something locally, then realize you want to undo that commit (and it was never pushed), you can use `git reflog` to find it and `git reset` to recover:

```
git reflog
```

This lists every recent position your branch pointer has been at, including commits that a later `reset` moved away from — each entry has a short hash. Find the commit you want back, then:

```
git reset --soft <commit-hash>
```

`--soft` moves your branch pointer to that commit but leaves all the file changes staged and intact — nothing is deleted, you're just "undoing" the commit boundary. **This is a last-resort tool, not a routine one** — reach for it only when you're specifically trying to recover something that already got committed, and always prefer `git restore` / `git restore --staged` (Sections 8.1–8.2) for anything simpler. Never run `git reset` on a branch anyone else has already pulled from (see Section 8.7).

8.4 Reverting an already-pushed commit

If a commit has already been pushed to a shared branch (most often: something bad got merged into `main`) and needs to be undone, **do not** try to rewrite history — use `git revert` instead:

```
git revert <commit-hash>
```

This creates a **new** commit that applies the exact opposite of the target commit, leaving the original commit in history untouched. Why this is preferred over rewriting shared history: anyone else who has already pulled the bad commit keeps a perfectly normal, conflict-free history — `git revert` only ever adds a new commit, it never deletes or rewrites one that others may already have. Rewriting a commit that's already on a shared branch (e.g. with `git reset --hard` + force-push) would desynchronize everyone else's local copy and require every teammate to manually fix their own clone — see Section 8.7.

8.5 Handling a broken `main`

Two options, and when to use each:

- **Revert the bad merge commit** (preferred for most cases): find the merge commit's hash on github.com (the PR page shows it) or via `git log main`, then `git revert -m 1 <merge-commit-hash>` (the `-m 1` tells Git which parent side of the merge is the "mainline" to revert back to — use `1`, which is `main` itself, for a normal PR merge). This is fast and immediately restores a working `main` while the real fix is figured out separately.
- **Roll forward with a fix** (better when the break is small and well-understood, and reverting would also remove other good changes bundled in the same merge): open a new `feature/*` branch from the current (broken) `main`, fix the specific problem, and merge that fix through the normal PR process.

Use revert when you need `main` working again *right now* and can sort out the real fix afterward; roll forward when the fix is quick, well-understood, and reverting would throw away unrelated good work from the same commit.

8.6 Re-deploying a known-good release

If a deploy goes out with a problem, you don't need to guess — redeploy from a commit you know was good:

```
git log --oneline main
```

Find the hash of the last commit you know worked correctly (e.g. the one before the problematic merge). On Render (Section 14 below), you can trigger a manual deploy of a specific commit from the service's **Deploys** tab (Render lists recent commits/deploys and lets you redeploy an earlier one), or push a `git revert` (Section 8.4) to `main` so the *next* automatic deploy picks up the fix. Prefer the `git revert`-then-redeploy path when possible, since it keeps "what's live" and "what's in `main`" in sync — manually redeploying an old commit while `main` has moved on is useful as an emergency stop-gap, but should be followed up with a proper revert or fix on `main` so the two don't stay out of sync.

8.7 What never to do on a shared branch, and why

Command	What it destroys	Why it's dangerous here
`git reset --hard <commit>`	Permanently discards all uncommitted changes in your working folder, and (if run on a branch others share) can silently drop commits from your local branch pointer	Run on a shared branch and then pushed, it can make it look like commits never happened, confusing or breaking every other teammate's local copy
`git clean -fd`	Permanently deletes every **untracked** file and folder in your working directory (anything not yet committed — including files Git never even knew about)	There is no undo. A half-finished file you hadn't committed yet, a `.env` you were still editing, a scratch script — all gone, with no reflog entry to recover it
`git push --force`	Overwrites the remote branch's history with your local history, discarding any commits on the remote that aren't in your local copy	On a shared branch like `main`, this can erase a teammate's already-pushed, already-reviewed work with no warning to them, and forces everyone else to manually repair their own local clone

These three are explicitly **not** part of this project's normal conflict-resolution or recovery process. Every situation in Sections 8.1–8.6 has a non-destructive answer. If you ever think you need one of these three, stop, re-read Sections 8.1–8.6 for the matching safe procedure first, and if you genuinely still believe a forced/destructive command is necessary, discuss it with the team before running it — not after.

---

9. File ownership and API contract management

9.1 Ownership, summarized

Document 1 Section 14 is the authoritative ownership matrix — refer back to it directly rather than treating any copy here as current. In short: Member 2 owns `client/src/**`; Member 3 owns `server/src/models/`, `server/src/routes/`, `server/src/controllers/`, `server/src/middleware/`, `server/src/config/`, `server/src/app.js`, `server/src/server.js`; Member 4 owns `server/src/providers/`, `server/src/jobs/`, `server/src/services/notifications/`; you own `server/src/services/priceComparison.js`, `priceMonitor.js`, `alertEvaluator.js`, plus `docs/` and `README.md`. Each owner's files are listed in that document alongside which files they must **not** touch without agreement first (e.g. Member 2 shouldn't edit anything under `server/**`; Member 4 shouldn't edit `priceComparison.js` / `priceMonitor.js`). "Ask before touching" means exactly that — a quick message before opening a PR that edits someone else's owned files, not a unilateral change discovered later in review.

9.2 Proposing a change to the shared API contract

Document 1 is the single source of truth for the API contract, schemas, and env vars. The process for changing any of it, in order, every time:

1. **The member proposes the change** — in a PR comment, a short written note (e.g. in the team's chat), or a comment directly on the relevant section if you're reviewing it live. State exactly what's changing and why (e.g. "I need `GET /api/monitor/status` to also return `lastRunTriggeredAlerts` so the dashboard can show it").
2. **You (Member 1) evaluate the impact** on the other three documents and on any code already written against the old contract — does this break Member 2's frontend calls, Member 3's route implementation, or Member 4's provider/notification code?
3. **Update Document 1 first.** The authoritative contract changes before anything else does.
4. **Update the affected role document(s)** (2, 3, and/or 4) to match.
5. **Only then** does the code change, in a normal feature-branch PR, referencing the doc update.

Never let step 5 happen before step 3 — code that silently drifts from Document 1 is exactly the "works on my machine" / contract-mismatch failure mode Section 18 below describes, and it compounds fast with four people editing different documents independently.

9.3 Check Document 4 for flagged additions

Document 4's author (Member 4) may flag small, necessary additions to Document 1 while building the Telegram integration — for example, a small addition Document 1 doesn't yet fully specify around the Telegram link-code endpoint's exact request/response shape used by the linking flow in Section 6.5. This is exactly the kind of thing Section 9.2's process exists for: a one- or two-field gap discovered mid-implementation, not a contract disagreement. On Day 1–2, check Document 4 for any such flagged items

and fold them into Document 1 promptly, following the steps above (Document 1 first, then the affected docs, then the code), so the contract stays the actual source of truth rather than quietly diverging from what Member 4 actually had to build.

---

10. Integration order

This restates Document 1 Section 15's cross-team summary concretely for the 7 days, per the brief's shared-contract-first sequence (Section 19 of the brief).

Day	What integrates	Who must wait on what
1	Contracts + scaffolding. Document 1 merged to `main`; `client/` and `server/` both start locally with `npm run dev`.	Nobody waits yet — everyone can scaffold their own area in parallel once Document 1 is merged.
2	Frontend and backend proceed **in parallel against the agreed contract**, using fixture data. Member 2 builds search/comparison UI against the documented response shapes even before Member 3's real routes exist, using fixture responses; Member 3 builds the real routes against the same documented shapes; Member 4 gets the fixture provider working.	Member 2 does not need to wait for Member 3's routes to be live — build against the documented contract and swap in the real backend once it exists. Member 4's fixture provider has no dependency and can start immediately.
3	Auth + DB integrated. Registration/login work end-to-end; dashboard, watchlist, and alert CRUD are wired to real routes, not fixtures, in the frontend.	Member 2's auth/dashboard/watchlist/alert UI now depends on Member 3's real routes being live — this is the first hard dependency point.
4	Comparison engine, monitoring orchestration, and alert evaluation integrated — **this is where your own code (`priceComparison.js`, `priceMonitor.js`, `alertEvaluator.js`) lands and gets tested against Member 3's `Offer`/`Alert` models and Member 4's provider interface for the first time.**	You depend on Member 3's finalized `Offer`/`PriceObservation`/`Alert` models and Member 4's provider dispatcher already existing and matching Document 1 Section 10.5 exactly. **This is the highest-risk integration point in the whole project** — three separately-built pieces meeting for the first time — and it must not be left until Day 4 night. Start a short, focused integration check with Member 3 and Member 4 as early as possible on Day 4, not after each of you has otherwise finished your Day-4 task list.
5	Notifications integrated end-to-end — a triggered alert produces a real email and Telegram message, both recorded in `NotificationEvent`.	Member 4's email/Telegram services depend on your `alertEvaluator.js` actually calling them correctly (matching the notification-service interface Document 4 defines); you depend on Member 4's services existing to test the full loop.
6	Stabilization + deployment (Sections 14–15 below).	Deployment depends on a genuinely working app from all three other members, not just your own services — don't start deployment with known-broken frontend or backend pieces.
7	Verify + demo (Sections 17, 20 below).	Nothing new integrates; this is verification against the deployed URLs and demo rehearsal.

---

11. Reviewing teammates' work — the PR checklist you actually use

Run through this on every PR you review, in this order:

1. **Does the endpoint/shape match Document 1 Section 6 exactly?** Method, path, auth requirement, request body fields, response envelope (`{success, data}` / `{success: false, error: {code, message}}`), and status code — character-for-character, not "close enough." A route returning `{token, user}` directly instead of `{success: true, data: {token, user}}` is a contract break even if it "basically works."
2. **Does the model match Document 1 Section 5 exactly?** Field names, types, required/optional, enums (e.g. `Offer.lastStatus` must only ever be `"ok"` / `"stale"` / `"error"` — nothing else), and indexes (e.g. the compound unique index on `{userId, offerId}` for `WatchlistItem`, `{provider, externalProductId}` for `Offer`).
3. **No secrets committed.** Scan the diff for anything that looks like an API key, token, password, or connection string typed directly into a file instead of read from `process.env`. Check that no `.env` file appears in the file list at all (GitHub's PR "Files changed" tab would show it if it did).
4. **No client-supplied `userId` bypassing auth.** Any controller touching `/api/watchlist`, `/api/alerts`, `/api/notifications`, `/api/notification-preferences` must scope its query to `req.user.id` (from the verified JWT), never to a `userId` taken from the request body or query string. This is the single most important security check in Document 1 Section 8, and the easiest kind of bug to miss in a quick skim.
5. **Centralized error handling used.** Errors should flow through the one `errorHandler.js` middleware (Document 1 Section 8), not be caught and formatted ad hoc inside individual controllers with a different error shape each time.
6. **Tests included and passing.** For backend changes, confirm there's at least a basic test for the new behavior and that `npm test` (Section 12 below) actually passes locally before you approve.
7. **No unrelated files swept in.** Check the PR's file list for anything that doesn't belong to the stated change — a tell-tale sign of a careless `git add -A` / `git add .` (Section 6.4). Ask the author to remove anything unrelated before merging.

---

12. Running tests before merges

12.1 Running the backend test suite locally

Before approving or merging any PR that touches `server/`, pull that branch and run the test suite yourself rather than trusting the PR description alone:

```
git fetch origin
  git switch <branch-name>
  cd server
  npm install
  npm test
```

(`npm install` picks up any new dependency the branch added; skip it if you know nothing changed in `package.json`.) Confirm every test passes before approving. If something fails, leave it as a review comment with the actual failure output rather than guessing at the cause.

12.2 A lightweight CI workflow (Day 6 polish, not a Day 1 requirement)

Once the team has basic velocity (treat this as nice-to-have polish for Day 6, not something that should block early feature work), add a GitHub Actions workflow that runs the backend test suite automatically on every PR. This is a **separate file** from `.github/workflows/monitor.yml` (which Document 4 defines for the scheduled monitoring trigger, Section 14.4 below) — keep these as two distinct workflow files, `test.yml` and `monitor.yml`, in `.github/workflows/`, so they don't get confused with each other.

Task: add CI test workflow

- **Files:** create `.github/workflows/test.yml`.
- **Commands:**

```
cd D:\Projects\PriceRadar
  git switch main
  git pull --ff-only origin main
  git switch -c feature/deployment
```

- **Implementation** — create `.github/workflows/test.yml` with this content:

```yaml
name: Backend Tests

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: server
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"
          cache-dependency-path: server/package-lock.json

      - name: Install dependencies
        run: npm ci

      - name: Run backend test suite
        run: npm test
```

- **Beginner explanation:** `on:` says this workflow runs whenever a pull request targets `main`, and also on every push directly to `main` (as a safety net). `runs-on: ubuntu-latest` means GitHub spins up a temporary Linux virtual machine to run the job — this costs nothing extra against your free Actions minutes beyond the job's actual runtime (a few seconds for this test suite). `defaults: run: working-directory: server` means every `run:` step below executes with `server/` as its current directory, so you don't have to `cd server` in every step manually. `actions/checkout@v4` clones your repo into that VM. `actions/setup-node@v4` installs Node 20 (matching Document 1 Section 3's chosen version) and caches `npm`'s download cache keyed on `server/package-lock.json`, which makes repeat runs faster. `npm ci` (not `npm install`) installs dependencies exactly as locked in `package-lock.json` — the right choice in CI, where you want a reproducible install, not one that might silently pick up a newer compatible version. `npm test` runs the Jest/Supertest suite Member 3 and you have been building.

- **Testing:** push this branch, open a PR, and confirm a new check named "Backend Tests" appears on the PR with a spinner, then a green checkmark once it passes (or a red X with logs if something's broken — click **Details** on the check to see the full output).
- **Expected result:** every future PR touching `server/` automatically shows pass/fail test results before anyone has to run `npm test` by hand to check.
- **Git workflow:** `feature/deployment` branch, PR reviewed by any teammate, squash-merge into `main` (Section 6.9).
- **Completion checklist:** file created at `.github/workflows/test.yml`; a test PR shows the check running and passing; `monitor.yml` (Section 14.4) is untouched and remains a separate file.

---

13. Environment variable management across the team

13.1 Keeping `.env.example` in sync

`server/.env.example` and `client/.env.example` (committed, placeholder values only — Document 1 Section 7) are the single source of truth for which env vars exist. Whenever anyone adds a new variable the code depends on, that PR must also update the relevant `.env.example` file in the same commit — a missing `.env.example` entry is exactly the kind of thing to flag in review (Section 11, item 2's spirit extends here: the contract isn't just the API, it's the environment too). Every teammate then copies `.env.example` to their own local `.env` (never committed) and fills in their own real values:

```
# in server/
  Copy-Item .env.example .env


  # in client/
  Copy-Item .env.example .env
```

13.2 Generating secrets

Both `JWT_SECRET` and `MONITOR_TRIGGER_SECRET` (Document 1 Section 7.1) need a long random string, generated the same way:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Run this once for `JWT_SECRET` and again (separately — do not reuse the same value) for `MONITOR_TRIGGER_SECRET`. Paste each result into your local `.env` for development; the production values (Section 14) are generated the same way but kept separate from your local dev values.

13.3 Where production env vars actually live

In production, every variable from Document 1 Section 7.1 and 7.2 is set directly in the hosting platform's dashboard — **Render's dashboard** for the backend's variables, and **Render's Static Site environment settings** for the frontend's `VITE_API_URL` (Section 14 below covers both). They are never committed to a file, matching Document 1 Section 7.3's rule exactly: `.env` is always git-ignored, `.env.example` holds placeholders only, and if a secret is ever accidentally committed, the fix is remove-from-file → commit the removal → **then rotate the actual credential** (a new Mongo password, a new JWT secret, a new API key) — deleting a line in a later commit does not remove it from git history, so the old value must be treated as compromised and replaced, not just hidden.

---

14. Deployment — full procedure

14.1 The decision, restated

Per Document 1 Section 12's verified findings: the **backend** deploys as a Render **free Web Service**, the **frontend** deploys as a Render **free Static Site**, the **database** is a **MongoDB Atlas free M0 cluster**, and the **scheduler** is a **GitHub Actions** scheduled workflow calling `POST /api/monitor/run`. This combination was chosen, not assumed, because Render's free web service **sleeps after 15 minutes of no inbound traffic** (waking takes roughly a minute on the next request) and Render has **no free tier for a dedicated, always-on Background Worker** — the cheapest worker instance is paid. Railway was considered and rejected because it no longer offers a renewing free tier (new accounts get a one-time usage credit, not an ongoing free allotment), which is a real risk of running out mid-project. GitHub Actions, by contrast, is genuinely free at this project's volume (private repos get 2,000 free minutes/month; this workflow uses well under a minute per run) — and because triggering it is just an HTTPS request to your backend, it conveniently **wakes** the sleeping Render service as a side effect, solving both "who runs the hourly schedule" and "the free tier sleeps" with one mechanism, instead of paying for an always-on worker.

14.2 Backend to Render

1. Create a free account at render.com (signing in with your GitHub account is the simplest option, and it's what lets Render list your repos directly).
2. Click **New +** → **Web Service**.
3. Connect your GitHub account if prompted, then select the `PriceRadar` repository (you may need to grant Render access to it specifically, since it's private).
4. **Name:** `priceradar-api` (or similar — this becomes part of the default URL, `https://priceradar-api.onrender.com`).
5. **Root Directory:** `server` — this tells Render the backend's `package.json` lives in `server/`, not the repo root.
6. **Build Command:** `npm install`
7. **Start Command:** `npm start` (confirm `server/package.json` has a `"start": "node src/server.js"` script — Member 3's setup should already include this per Document 3).

8. **Instance Type:** select **Free**.
9. Under **Environment Variables**, add every variable from Document 1 Section 7.1 — `PORT`, `NODE_ENV` (set to `production`), `MONGODB_URI` (your real Atlas connection string, Section 14.4 below), `JWT_SECRET`, `JWT_EXPIRES_IN`, `CLIENT_ORIGIN` (a **placeholder** for now, e.g. `http://localhost:5173` — you'll update this once the frontend has a real URL, Section 14.3), `FLIPKART_AFFILIATE_ID` / `FLIPKART_AFFILIATE_TOKEN` (blank unless Flipkart approval has actually arrived, per Document 1 Section 10.1), `BREVO_API_KEY`, `EMAIL_FROM`, `TELEGRAM_BOT_TOKEN`, `MONITOR_TRIGGER_SECRET`, `MONITORING_ENABLED` (`true`).
10. Click **Create Web Service**. Render builds and deploys automatically; watch the build logs in the dashboard.
11. Once deployed, verify the health check against the real public URL:

```
curl https://priceradar-api.onrender.com/api/health
```

Expect `200` with a body like `{"success":true,"data":{"status":"ok","time":"2026-10-19T08:00:00.000Z"}}` (matching Document 1 Section 6.6's shape). This is ACC-07 from Document 1 Section 17.

14.3 Frontend to Render

1. From the Render dashboard, **New +** → **Static Site**.
2. Select the same `PriceRadar` repository.
3. **Name:** `priceradar-web` (becomes `https://priceradar-web.onrender.com`).
4. **Root Directory:** `client`
5. **Build Command:** `npm run build`
6. **Publish Directory:** `dist` (Vite's default production build output folder).
7. Under **Environment Variables**, add `VITE_API_URL` set to your backend's real Render URL from Section 14.2 (e.g. `https://priceradar-api.onrender.com`).
8. Click **Create Static Site**. Once deployed, open the given URL in a browser and confirm the site loads (even if some features still fail due to CORS — that's expected until the next step).

14.4 Closing the loop: update `CLIENT_ORIGIN`

There's a chicken-and-egg order here: the backend needs the frontend's real URL for CORS, but the frontend's real URL doesn't exist until *after* it's deployed. Resolve it in this order:

1. Deploy the backend first with a **placeholder** `CLIENT_ORIGIN` (Section 14.2, step 9) — the backend will run fine, just with CORS not yet correctly configured for the real frontend.
2. Deploy the frontend (Section 14.3) and note its real `https://priceradar-web.onrender.com` URL.
3. Go back to the backend's Render dashboard → **Environment** tab, update `CLIENT_ORIGIN` to the frontend's real URL, and save — Render automatically redeploys/restarts the service with the new value. If it doesn't restart automatically, use the **Manual Deploy** button to force a restart.

14.5 MongoDB Atlas network access

Confirm **Network Access** in the Atlas dashboard still allows `0.0.0.0/0` (Document 1 Section 8) — this was already set during local development and should be left as-is for this deployment. Reasoning, stated plainly: Render's free-tier outbound IP addresses are not static/published, so there is no fixed IP range to allowlist instead — `0.0.0.0/0` (allow from anywhere) combined with a strong, unique database password is the pragmatic choice for a 7-day training project on free-tier infrastructure, not a production-grade setup, and Document 1 already documents this trade-off explicitly rather than hiding it.

14.6 Wiring up the scheduled monitoring workflow

Document 4 already writes `.github/workflows/monitor.yml` (its content is Member 4's responsibility, referencing `BACKEND_URL` and `MONITOR_TRIGGER_SECRET` as repo secrets). Your job here is wiring in the **real values** as GitHub repo secrets:

1. On github.com, go to the repo → **Settings** → **Secrets and variables** → **Actions**.
2. Click **New repository secret**.
3. Add `BACKEND_URL` = your backend's real Render URL (e.g. `https://priceradar-api.onrender.com`, no trailing slash).
4. Add `MONITOR_TRIGGER_SECRET` = the exact same value you set as the `MONITOR_TRIGGER_SECRET` environment variable on the Render backend (Section 14.2, step 9) — these two must match character-for-character, since the workflow sends this value as the `X-Monitor-Secret` header and the backend compares it against its own copy (Document 1 Section 6.6).

14.7 Triggering the workflow manually and verifying it actually ran

Per the brief's instruction to actually verify this, not assume it:

1. Go to the repo's **Actions** tab, select the **monitor.yml** workflow from the left sidebar (it should appear once `monitor.yml` exists on `main`, since `workflow_dispatch` — a manual trigger button — needs to be defined in the file for this button to appear; confirm with Member 4 that `on: workflow_dispatch:` is included alongside the `cron:` schedule).
2. Click **Run workflow** → **Run workflow** (the dropdown defaults to running against `main`).
3. Wait for the run to complete, then click into it and check the step logs — confirm it actually made the HTTP request and got a success response back (e.g. `202` with `{started: true, offersChecked: n, triggeredAlerts: n}`, per Document 1 Section 6.6).
4. Cross-check in Render: open the backend's **Logs** tab and confirm a corresponding `POST /api/monitor/run` request actually arrived and was processed around the same timestamp — don't rely on the Actions log alone, since that only proves the request was *sent*, not that the backend actually *received and processed* it. Seeing both sides match is what makes this ACC-08 (Document 1 Section 17) actually verified rather than assumed.

---

15. Distinguishing the 5 separate deployment checks

Per the brief's explicit instruction, these are checked **independently** — passing one is never assumed to mean another also passes.

Check	What "pass" means	How to verify
Deployed frontend	The static site loads in a browser	Open `https://priceradar-web.onrender.com` directly and confirm the page renders
Deployed API	The backend process is up and responding	`GET /api/health` returns `200 {status:"ok"}` from the public Render URL (not localhost)
Connected production database	The backend can actually read/write Atlas, not just that `MONGODB_URI` is set	Register a real test user against the live URL and confirm it appears in Atlas's collection browser
Functioning background scheduler	GitHub Actions is actually invoking the backend on schedule, not just configured to	An Actions run log (scheduled, not only manually triggered) shows a successful `POST /api/monitor/run`, cross-checked against the Render backend logs (Section 14.7)
Verified live retailer integration	A real, non-sample price was actually fetched from an approved live source	**Only** true if Flipkart affiliate approval has actually arrived and a real priced offer has been fetched and logged — otherwise, state explicitly: **"Not yet verified — demo runs on fixture data,"** per Document 1 Section 10.6's honesty requirement. Do not infer this from the other four checks passing; it depends entirely on Flipkart's own approval timeline.

---

16. Post-deployment test checklist

Work through this against the **deployed URLs**, not localhost, before declaring deployment complete:

1. **Register and log in** with a fresh test account against the live frontend URL.
2. **Search and compare** — confirm a known fixture product returns with at least two labeled offers (ACC-01).
3. **Create an alert** (target-price or fixed-drop) on a watched offer.
4. **Manually trigger monitoring** (`POST /api/monitor/run` with the correct `X-Monitor-Secret`, via Postman or the GitHub Actions `workflow_dispatch` button, Section 14.7) and confirm a new `PriceObservation` was actually written — check via Atlas's collection browser or the offer's history endpoint (`GET /api/offers/:offerId/history`).
5. **Test email delivery** to a real inbox you control — lower a fixture price below your alert's threshold, re-trigger monitoring, and confirm the email actually arrives (check spam too, the first time).
6. **Test Telegram delivery** to a real linked chat, the same way.
7. **Check Render's server logs** for unexpected errors during all of the above — the **Logs** tab on each Render service.

8. **Verify a deliberately-stale/failed offer still shows correctly** in the UI — temporarily force a fixture provider entry to fail (per Document 4's testing guidance), re-run monitoring, and confirm the offer shows an "error"/"stale" state rather than a price of ₹0 or a false alert (ACC-06).
9. **Test the whole flow from a fresh/incognito browser window** — register → search → watchlist → alert → logout → login again — to catch any place the app silently relies on leftover `localStorage` or cookies from your regular dev browser session.
10. **Document the final deployed URLs** (frontend and backend) in `README.md`, replacing the "To be filled in on Day 6" placeholder from Section 3.4.

---

17. Release checklist and demo preparation

17.1 Pre-demo checklist

- Every ACC item in Document 1 Section 17 passes **against the deployed URLs** specifically, not just locally (cross-reference Section 20 below, which restates the full ACC table as your personal sign-off).
- The team has agreed, out loud, on exactly which data in the demo is live and which is sample — and will say so plainly if asked, with no exceptions under demo-day pressure (Document 1's non-negotiable honesty rule, Section 1.3).
- Each team member can explain their own area of the system in under two minutes, in their own words — not reading from this document.

17.2 Rehearsed demo script

1. **Register** a new user on the live frontend.
2. **Search** for a known fixture product.
3. **View the comparison page** — point out the "Lowest price" label and the "Sample data" badges where applicable.
4. **Add an offer to the watchlist.**
5. **Create an alert** (target-price or fixed-drop) on that offer.
6. **Manually trigger monitoring** via the GitHub Actions `workflow_dispatch` button (Section 14.7) — narrate what's happening while it runs.
7. **Show the notification arriving** — email inbox and/or Telegram chat, live.
8. **Show the notification history page** reflecting the same event.
9. **Show the GitHub Actions run log** as concrete evidence of real, working scheduled execution — not just a claim.

---

18. Common team coordination failures and how to avoid them

- **Two members editing the same shared file without coordinating** — e.g. both Member 3 and you touching `server/src/models/Alert.js` in the same window. Avoid it by checking Document 1 Section 14's "do not edit without agreement" column before starting any change to a file you don't primarily own, and by posting in the team chat before you start ("touching `Alert.js` to add X, will have a PR up in ~1hr").
- **A frontend member building against a guessed API shape** instead of Document 1's actual documented contract. Avoid it by treating Document 1 as the only source of truth for request/response shapes — if the contract seems to be missing something needed, that's a Section 9.2 proposal, not a reason to improvise a shape and hope it matches later.
- **A backend change that silently breaks Document 1's contract** without updating the doc first — e.g. renaming a response field because it "reads better," without going through Section 9.2. Avoid it by treating any PR that changes a route's request/response shape, without a matching Document 1 update in the same PR or a prior one, as an automatic "request changes" in review.
- **Secrets accidentally committed.** If it happens: remove the secret from the file, commit the removal, **then rotate the actual credential immediately** (generate a new value and update it everywhere it's used — local `.env` files, the Render dashboard, GitHub Actions secrets) — per Document 1 Section 7.3, deleting the line in a later commit does not remove it from git history, so treat the old value as permanently compromised.
- **A merge to `main` that isn't tested before the next person builds on top of it.** Avoid it with the habit from Section 6.10: after pulling any merge to `main`, actually run the app (or the relevant test) before starting new work on top of it, every time — not just "it probably still works."

---

19. Troubleshooting

Render deploy fails during build. Open the **Logs** tab on the failing deploy. A very common cause is a Node version mismatch between what Render's default build image uses and what the project expects — pin it explicitly by adding this to both `server/package.json` and `client/package.json`:

```
"engines": {
    "node": "20.x"
  }
```

CORS errors in production (browser console shows something like `has been blocked by CORS policy`). Almost always a `CLIENT_ORIGIN` mismatch on the backend — confirm it's set to the frontend's *exact* real Render URL (Section 14.4), including `https://` and no trailing slash, and that the backend actually restarted after you changed it (Render should auto-redeploy on an env var change; if unsure, trigger a Manual Deploy).

MongoDB Atlas connection refused in production. Check **Network Access** in the Atlas dashboard (Section 14.5) — confirm `0.0.0.0/0` is still present and not expired (Atlas lets you set a temporary access-list entry with an expiry, which is easy to accidentally create instead of a permanent one). Also double-check `MONGODB_URI` in Render's environment variables has the correct username/password and database name, with no leftover `<password>` placeholder text.

GitHub Actions workflow failing. Open the failing run on the **Actions** tab and read the actual step logs — don't guess. Common causes: a mistyped secret name (`BACKEND_URL` vs `BACKEND_UR`, case matters), a wrong/stale URL in the `BACKEND_URL` secret, or the backend being asleep and the request timing out before Render finishes waking it up. Render's free tier cold-start can take **30–60 seconds** after 15 minutes of inactivity — if `monitor.yml`'s HTTP request step uses a short timeout (e.g. `curl`'s default), it may need a more generous timeout value (e.g. `curl --max-time 90 ...`) to tolerate a cold start rather than failing the whole run; raise this with Member 4 if the workflow is timing out specifically on cold backend instances.

---

20. Final acceptance criteria

The project is releasable for the Day 7 demo when every item below passes **against the deployed URLs**. This restates Document 1 Section 17's ACC-01 through ACC-10 as your personal final sign-off checklist.

ID	Area	Check	How I verify this
ACC-01	Search	Searching a known fixture product name returns it with at least 2 labeled offers	Search the live frontend for a fixture product name, confirm ≥2 offer rows with provider labels
ACC-02	Comparison	The lowest valid offer is text-labeled "Lowest price," not color-only	Open the comparison page, visually confirm the text label (not just a color highlight) on the cheapest valid row
ACC-03	Auth	A second registered user cannot GET another user's `/api/watchlist` or `/api/alerts`	Register two test accounts, log in as the second, attempt the first's data via Postman with the second's JWT, confirm `403`/empty results, never the first user's data
ACC-04	Monitoring	A fixture price edited below a target-price alert's threshold, followed by `POST /api/monitor/run`, results in exactly one `NotificationEvent` with `status: "sent"` per channel	Edit a fixture price, create a matching alert, trigger monitoring (Section 14.7), check `/api/notifications` for exactly one `"sent"` event per selected channel
ACC-05	Dedup	Calling `/api/monitor/run` again immediately after, price unchanged, produces no new `"sent"` events for that alert	Trigger monitoring a second time right after ACC-04, confirm no additional `"sent"` event (and ideally a `"skipped_duplicate"` one if it ran in the same hour)
ACC-06	Failure handling	A provider forced to fail results in `Offer.lastStatus = "error"`, not a price of 0 and not a false alert	Force a fixture entry invalid (Document 4's test method), trigger monitoring, confirm `lastStatus: "error"`, the prior price unchanged, and no alert fired
ACC-07	Deployment	`GET /api/health` returns `200 {status:"ok"}` from the public backend URL	`curl` the live `/api/health` URL directly (Section 14.2, step 11)
ACC-08	Scheduler	The GitHub Actions workflow run log shows at least one successful `POST /api/monitor/run` call against the deployed backend	Check an Actions run log *and* the matching Render backend log entry (Section 14.7)
ACC-09	Secrets	`git log -p -- '**/.env'` on the repo shows no `.env` file was ever committed	Run `git log -p -- '**/.env'` from the repo root locally and confirm empty output
ACC-10	Honesty	Every `isSample: true` offer visibly shows a "Sample data" badge in the UI	Visually check every sample-labeled offer on the live comparison page for the badge

The project is truly done only when every row above is checked **against the live, deployed system** — not from memory, not "it worked locally once" — and the team can truthfully state, for each data source shown in the UI, whether it's live or sample, with no exceptions even under demo-day pressure (Document 1 Section 19).